

Architecture Comparison

Downloaded Argus Security 3.0 versus the desktop Project ARGUS Extension 5.1

Core architectural shift

The downloaded build is primarily a score-driven deterministic rule engine. The desktop build is a layered, lifecycle-aware, evidence-governed hybrid detection system in which policy - not the model - controls the user-visible status, score, and warning.

Downloaded source	Desktop source
C:/Users/User/Downloads/Argus-Security-main/Argus-Security-main	C:/Users/User/Desktop/Project-Argus-Extension
Manifest version 3.0.0	Manifest version 5.1.0

1. System flow and authority

Downloaded Argus Security 3.0	Desktop Project ARGUS 5.1
Page metadata + webRequest	Page, network, and security metadata
DOWN	DOWN
service_worker.js	Navigation and session validation
DOWN	DOWN
argus_engine or legacy fallback	Modular evidence engine + local trained model
DOWN	DOWN
Numeric score threshold	Evidence decision policy
DOWN	DOWN
Warning + popup + storage	Policy-approved score + warning permissions + UI/storage/reports

Who has final authority?	Why it matters
3.0: the combined numeric score and thresholds directly drive SAFE, SUSPICIOUS, HIGH_RISK, and warning behavior.	A high score can become a visible warning even when the underlying signals are mainly static or contextual.
5.1: the evidence decision policy is the sole user-visible authority. The model is advisory and cannot independently create SUSPICIOUS, HIGH_RISK, or a warning.	The UI receives a policy-approved score and explicit warningAllowed/overlayAllowed permissions, reducing model-only score inflation.

2. What is different - detection and scoring

Architecture area	Downloaded Argus 3.0	Desktop Project ARGUS 5.1
Primary decision model	Weighted deterministic analyzers feed a combined numeric score. The UI and warning logic largely follow thresholds.	A hybrid design combines modular evidence analysis with a local trained calibrator, then applies a separate evidence policy before anything becomes user-visible.
Risk states	SAFE, SUSPICIOUS, HIGH_RISK.	SAFE, MONITORING, UNCERTAIN, SUSPICIOUS, HIGH_RISK. Observation quality is represented without treating uncertainty as danger.
High-risk rule	HIGH_RISK at a score of 70 or above.	HIGH_RISK requires allowlisted direct evidence connecting a sensitive source or dangerous action to a harmful sink.
Suspicious rule	SUSPICIOUS at a score of 35 or above.	SUSPICIOUS requires correlated evidence rather than score alone: at least two evidence groups and at least one observed behavioral group in the inspected policy.
Model-only output	No independent governance layer prevents a high combined score from reaching the UI.	Model-only output is capped at 5, cannot warn, and cannot independently create SUSPICIOUS or HIGH_RISK.
Static/context signals	Static and contextual features can contribute directly to the overall score.	Static single-group evidence is capped at 10. Context features inform confidence and analysis but are restricted from independently inflating visible risk.
Incomplete observation	Limited explicit separation between incomplete telemetry and actual risk.	Incomplete observation lowers confidence or yields MONITORING/UNCERTAIN; it is not supposed to add risk by itself.

3. What is different - lifecycle, destinations, and UI

Architecture area	Downloaded Argus 3.0	Desktop Project ARGUS 5.1
Navigation isolation	Primarily tab-based maps, timer-based resets, and preservation logic.	navigation_session_guard.js tracks tabId, pageKey, epoch, navigationId, frameId, and rejects stale messages or rescans.
SPA behavior	Limited lifecycle treatment beyond fixed content-script scans.	Explicit same-document page change handling for SPA navigation, plus stale page-context detection.
Scan lifecycle	Initial scan followed by fixed rescans around 1.5 and 4 seconds.	Progressive preliminary, final, and interaction-final phases with observation windows and shadow-mode support.
Destination classification	Mostly same-site, third-party, and insecure destination distinctions.	Fine-grained roles include first-party write, same-site API, identity/SSO, payment, analytics, ad network, CDN, unknown read/write/beacon, and executable source.
Popup freshness	Popup reads the cached result immediately.	Popup requests a rescan and checks same page, post-request freshness, and stable scan state before rendering. It also exposes richer evidence and export diagnostics.
Analyzer surface	Seven analyzers plus a combiner form the core engine.	Adds browser protection, URL lexical, mixed content, security headers, script integrity, unsandboxed frames, storage/cookie relay, WebSocket relay, encoded relay, and wildcard postMessage analysis.
Testing and validation	Three primary test files and deterministic checks.	Sixteen primary test files plus Playwright real-browser testing, datasets, training, randomized cross-validation, benign robustness tests, and an autonomous controller.
Versioned contracts	Simpler manifest-centered versioning.	Separate manifest, policy, model, evidence-policy, and report-schema versions. This improves traceability but creates drift risk if releases are not synchronized.

4. What is still the same

Shared architecture element	What remains common
Extension platform	Both are Chrome Manifest V3 extensions.
Major runtime pieces	Both use content.js, service_worker.js, popup/options pages, and local extension storage.
Network observation	Both use chrome.webRequest metadata as a central network signal source.
Privacy posture	Both are designed around metadata rather than request bodies, typed values, credentials, or personal content.
Local operation	Both operate locally and do not depend on an external generative AI service for normal scoring.
Backend role	Both include an optional FastAPI compatibility/backend path; the browser extension remains the main runtime authority.
Fallback behavior	Both retain a legacy fallback scorer when the modular engine is unavailable.
Detection foundation	Both retain analyzer-based evidence collection and score combination as important implementation building blocks.
Product shell	Permissions, popup workflow, service-worker coordination, and the overall extension product shape remain broadly recognizable.

Measured implementation growth

Item	Downloaded	Desktop	Growth
content.js	42,335	50,002	1.18x
service_worker.js	52,482	74,843	1.43x
popup.js	17,301	27,109	1.57x
engine/argus_engine.js	31,581	41,391	1.31x
backend/main.py	15,004	17,671	1.18x
Primary test files	3	16	5.33x

Byte counts are a structural indicator, not a quality score. The largest relative growth is in popup.js and the test suite, reflecting richer diagnostics, freshness checks, and validation coverage.

5. Architectural consequences and remaining risks

Improved separation of concerns	Evidence collection, model calibration, lifecycle validation, and UI policy are separate stages. This makes it easier to diagnose whether a wrong score came from telemetry, classification, the model, policy, or display mapping.
Safer user-visible decisions	The desktop design explicitly prevents model-only scores and static complexity from directly producing warnings. Direct evidence is required for HIGH_RISK, while correlated evidence is required for SUSPICIOUS.
Better browser lifecycle handling	Session IDs, document/page keys, epochs, frame checks, and progressive scans reduce stale state surviving navigation - a major source of misleading popup results in browser extensions.
More operational complexity	The stronger architecture has more modules, contracts, report fields, and version identifiers. It needs synchronized releases and end-to-end browser tests to avoid schema or version drift.

Remaining risks

Remaining engineering risk	Why it remains important
Monolithic content.js and service_worker.js	Both files are still large coordination hubs, which makes lifecycle and state interactions harder to reason about.
Legacy fallback and backend scorer drift	Alternative scoring paths can diverge from the modular evidence policy unless parity is tested explicitly.
Multiple version identifiers	Manifest 5.1.0, policy 4.2.0, model 5.0.0, evidence policy v1, and report schema 2 improve traceability but require disciplined compatibility checks. Some documentation still refers to 4.2.
Configuration duplication	Provider and category knowledge appears in both code and JSON configuration in places, increasing the chance of inconsistent destination classification.

6. Bottom line

Project ARGUS 5.1 is not merely a larger version of Argus 3.0. It changes the decision architecture: browser telemetry is validated against navigation state, evidence is grouped and classified, a local model is constrained to an advisory role, and an explicit policy determines the user-visible result.

Question	Answer
Is the extension shell the same?	Broadly yes. Both use the same MV3 content-script/service-worker/popup pattern and local metadata collection.
Is the detection architecture the same?	No. The desktop build adds explicit lifecycle isolation, evidence governance, constrained model calibration, richer destination roles, and progressive observation.
Is the desktop version safer against score inflation?	Architecturally yes: model-only and context-only paths are constrained. Real-browser validation is still required to prove every runtime path follows that policy.
Can the old code be considered a separate product lineage?	It is the same product lineage and shell, but the desktop build represents a substantial decision-system redesign rather than a simple feature update.

Review basis

This comparison was prepared from the source trees, manifests, engine modules, service-worker and popup flows, backend compatibility code, tests, and architecture documentation present in the two local project directories. No files were modified in the downloaded Argus source.